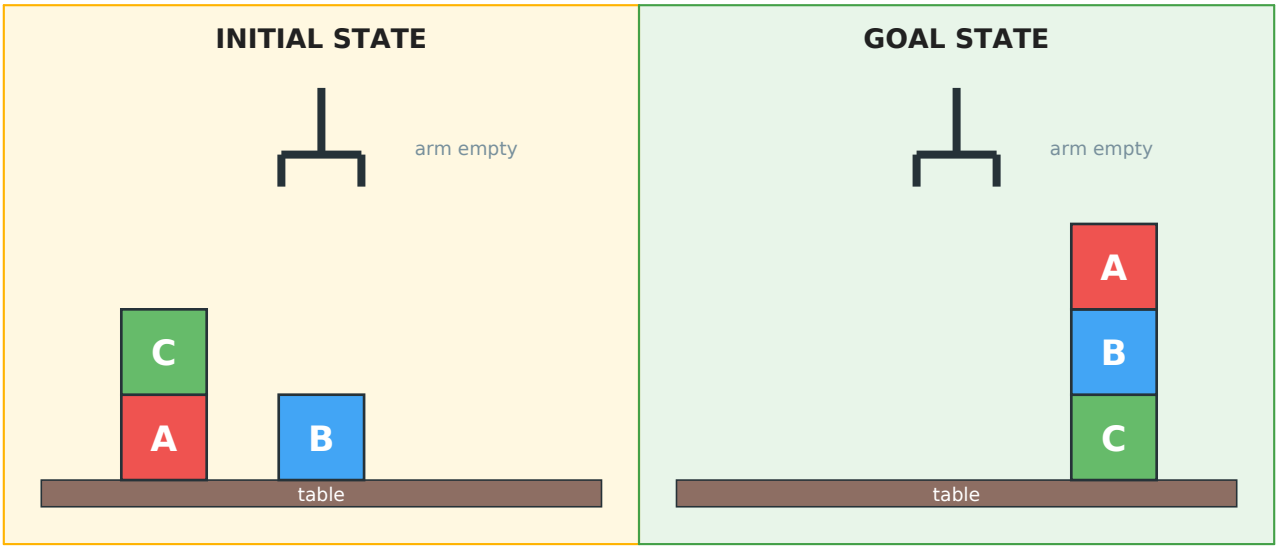
Problem 5: Blocks World

Level: **Basic / Intermediate** · Folder: **05_Blocks_World/** · Files: domain.pddl, problem.pddl, solution_plan.txt

1. The problem

Three blocks A, B and C are on a table. Block C is on top of A, and B is on the table. A robot arm can hold one block at a time. The goal is a single tower with **A on B** and **B on C**. This instance is the famous **Sussman anomaly**.



The goal picture shows one state that satisfies the goal (the final state of the plan). Only the facts in the :goal section are required; anything else may be different.

What you will learn

- The best-known planning benchmark and its four classic actions.
- The idea of **clear** (nothing on top) and of the arm state (*handempty* / *holding*).
- **Interacting subgoals**: solving one goal first can undo the other one.

2. Formulating the problem in PDDL

Step 1: Types and objects. One type `block`; objects A, B, C.

Step 2: Predicates describe what can be true or false in a state:

Predicate	Meaning
`(on ?x ?y)`	Block ?x is directly on block ?y.
`(ontable ?x)`	Block ?x is on the table.
`(clear ?x)`	No block is on ?x (and the arm is not holding it).
`(handempty)`	The arm holds nothing.
`(holding ?x)`	The arm holds block ?x.

Step 3: Actions. Each action has *parameters*, a *precondition* (what must be true to apply it) and an *effect* (facts that become **true (+)** or **false (−)**):

Action	Precondition	Effect

pick-up
`?x - block`
Lift a clear block from the table.
	`(clear ?x)`
	`(ontable ?x)`
	`(handempty)`
		`+ (holding ?x)`
		`− (ontable ?x)`
		`− (clear ?x)`
		`− (handempty)`

put-down
`?x - block`
Put the held block on the table.
	`(holding ?x)`	`+ (ontable ?x)`
		`+ (clear ?x)`
		`+ (handempty)`
		`− (holding ?x)`

stack
`?x - block, ?y - block`
Put the held block ?x onto clear block ?y.
	`(holding ?x)`
	`(clear ?y)`
		`+ (on ?x ?y)`
		`+ (clear ?x)`
		`+ (handempty)`
		`− (holding ?x)`
		`− (clear ?y)`

unstack
`?x - block, ?y - block`
Lift clear block ?x from the top of block ?y.
	`(on ?x ?y)`
	`(clear ?x)`
	`(handempty)`
		`+ (holding ?x)`
		`+ (clear ?y)`
		`− (on ?x ?y)`
		`− (clear ?x)`
		`− (handempty)`

Step 4: Initial state (`:init`) lists *every* fact that is true at the start. Everything else is false (closed-world assumption). **Step 5: Goal** (`:goal`) lists the facts that must be true at the end.

INITIAL STATE (6 facts)	GOAL

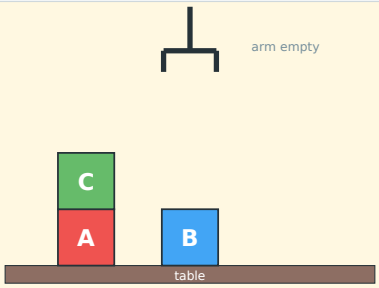
`(on C A)`
`(ontable A)`
`(ontable B)`
`(clear C)`
`(clear B)`
`(handempty)`
	`(on A B)`
	`(on B C)`

Modelling notes

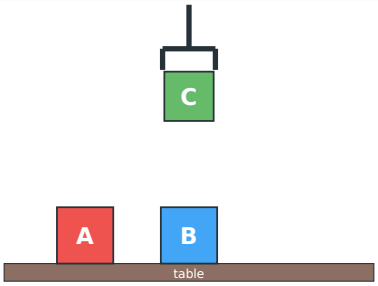
- **Why is it an 'anomaly'?** Suppose you work on one goal at a time. If you first build (on A B), you must take it apart again to put B on C. If you first build (on B C), block A is trapped under C and B. A correct plan has to *interleave* the two goals.
- The goal is **partial**. It does not say (ontable C), but that follows anyway.
- pick-up/put-down handle the table. stack/unstack handle another block. This is because the table is not a block and is always 'clear'.

3. Solution plan (6 actions, optimal)

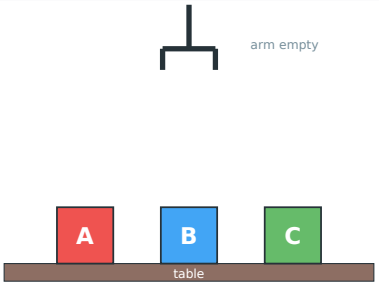
The optimal plan is 6 steps: first clear A by moving C to the table, then build the tower from the bottom up. The pictures below show the state **after** each action. They were produced by executing the plan on the PDDL model.



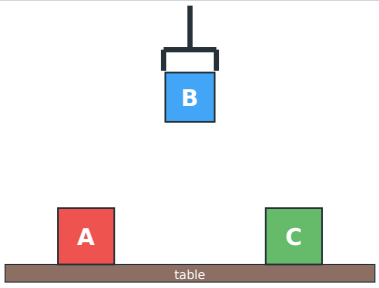
Initial state



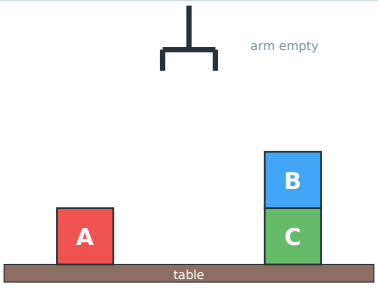
Step 1
`(unstack c a)`



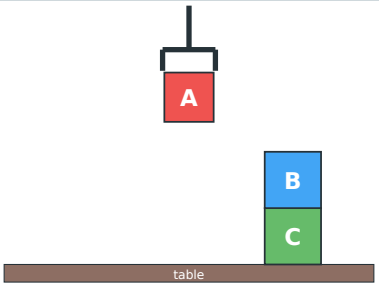
Step 2
`(put-down c)`



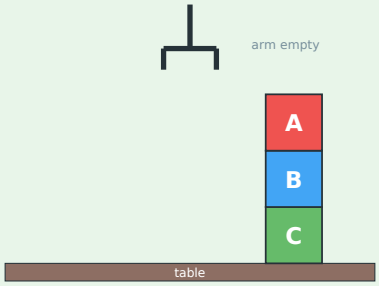
Step 3
`(pick-up b)`



Step 4
`(stack b c)`



Step 5
`(pick-up a)`



Step 6
`(stack a b)`

4. How to run it

Online (no installation)	Open **editor.planning.domains**, paste *domain.pddl* and *problem.pddl* into two tabs and press **Solve**.
pyperplan (Python)	`pip install pyperplan` `pyperplan domain.pddl problem.pddl` (breadth-first search) `pyperplan -s astar -H lmcut domain.pddl problem.pddl` (optimal A*)
Fast Downward	`fast-downward.py domain.pddl problem.pddl --search "astar(lmcut())"`

5. Lab exercises

1. Four blocks: start with the tower D-C-B-A (A on the table) and reverse it.
2. Write the goal as a complete state, including the ontable and clear facts. Does the plan change?
3. Remove the **handempty** precondition from pick-up. What goes wrong in the plans?

6. PDDL source code (appendix)

domain.pddl

```
;; ============================================================
;; Problem 5 : Blocks World
;; Level     : Basic / Intermediate
;; A robot arm stacks and unstacks blocks on a table.
;; ============================================================
(define (domain blocks-world)
  (:requirements :strips :typing)

  (:types block)

  (:predicates
    (on ?x - block ?y - block)  ; block ?x is directly on block ?y
    (ontable ?x - block)        ; block ?x is on the table
    (clear ?x - block)          ; nothing is on top of ?x
    (handempty)                 ; the arm holds nothing
    (holding ?x - block)        ; the arm holds block ?x
  )

  ;; Lift a clear block from the table
  (:action pick-up
    :parameters (?x - block)
    :precondition (and (clear ?x) (ontable ?x) (handempty))
    :effect (and (holding ?x)
                 (not (ontable ?x)) (not (clear ?x)) (not (handempty)))
  )

  ;; Put the held block on the table
  (:action put-down
    :parameters (?x - block)
    :precondition (holding ?x)
    :effect (and (ontable ?x) (clear ?x) (handempty)
                 (not (holding ?x)))
  )

  ;; Put the held block ?x on top of a clear block ?y
  (:action stack
    :parameters (?x - block ?y - block)
    :precondition (and (holding ?x) (clear ?y))
    :effect (and (on ?x ?y) (clear ?x) (handempty)
                 (not (holding ?x)) (not (clear ?y)))
  )

  ;; Lift a clear block ?x from the top of block ?y
  (:action unstack
    :parameters (?x - block ?y - block)
    :precondition (and (on ?x ?y) (clear ?x) (handempty))
    :effect (and (holding ?x) (clear ?y)
                 (not (on ?x ?y)) (not (clear ?x)) (not (handempty)))
```

```
  )
)
```

problem.pddl

```
;; The "Sussman Anomaly"
;; Initial:  C is on A; A and B are on the table.
;; Goal   :  A on B, B on C  (a single tower A-B-C).
(define (problem sussman-anomaly)
  (:domain blocks-world)

  (:objects A B C - block)

  (:init
    (on C A)
    (ontable A)
    (ontable B)
    (clear C)
    (clear B)
    (handempty)
  )

  (:goal (and (on A B)
              (on B C)))
)
```

solution_plan.txt

1. (unstack c a)
2. (put-down c)
3. (pick-up b)
4. (stack b c)
5. (pick-up a)
6. (stack a b)